

**Budowa aplikacji webowych w ASP.NET Core
z wykorzystaniem EntityFramework Core, Migracje
i wzorzec MVC w praktyce**



Imię i nazwisko: Aleksander Chomicz

Numer albumu: 157523

Kierunek studiów, semestr, rok akademicki:

Informatyka i inżynieria danych, stacjonarne, II stopień, II semestr, 2025/2026

1. Cel

Celem dzisiejszych zajęć jest pokazanie w jaki sposób zacząć tworzyć taką aplikację webową „od zera” oraz jak w nowoczesny sposób zarządzać bazą danych w projekcie, jak rozbudować istniejącą aplikację o nową funkcjonalność np. **(Zgłaszanie Szkody Samochodu)** w przykładowym już istniejącym projekcie *CarRentalSystem*, wykorzystując mechanizmy automatyzacji Visual Studio 2022 (Scaffolding).



2. Porównanie technologii (dlaczego taki wybór?)

Cecha	ASP.NET (Framework)	ASP.NET Core
Platforma	Tylko Windows.	Wieloplatformowy: Windows, macOS, Linux.
Architektura	Stos oparty na .NET Framework.	Modułowy, zbudowany na nowym stosie.
Wydajność	Dobra, ale starsza architektura.	Znacznie wyższa wydajność, zoptymalizowany.
Hosting	Głównie IIS (Internet Information Services).	Elastyczny: IIS, Nginx, Docker, chmura (Kestrel).
Model Programowania	Oddzielne ASP.NET MVC i Web API.	Zunifikowany: łączy MVC i Web API.
Open Source	Nie, głównie zamknięty kod.	Tak, otwarty kod (Open Source).
Zastosowanie	Starsze aplikacje, projekty oparte na Windows.	Nowe projekty, aplikacje chmurowe, mikroustugi.

Cecha	Entity Framework 6 (EF6)	Entity Framework Core (EF Core)
Platforma	.NET Framework (Windows)	.NET Core/.NET 5+ (Multiplatform)
Wydajność	Dobra, ale wolniejsza	Znacznie szybsza i zoptymalizowana
Status	Stabilny, nierozwijany	Aktywnie rozwijany, przyszłość .NET
Architektura	Monolityczna	Lżejsza, modułowa (NuGet packages)
Bazy danych	Głównie SQL Server	Szerokie wsparcie (wiele providerów)

3. Najczęstsze problemy po pobraniu projektu na inne urządzenie

„U mnie działa, a u Ciebie nie ma bazy”



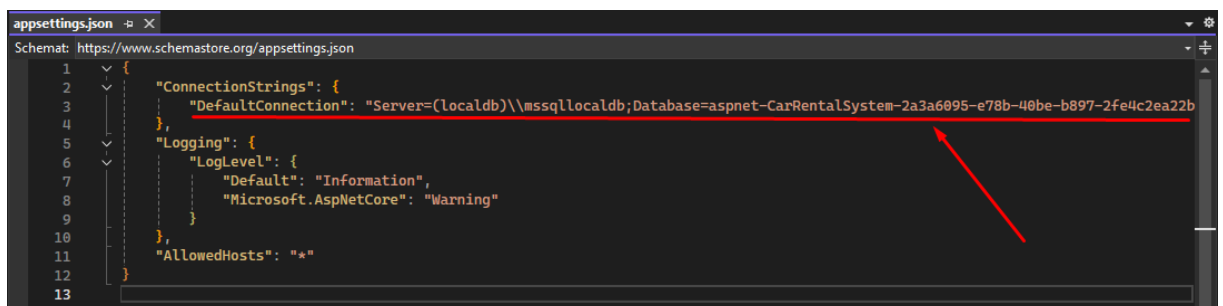
Kolejny problem z migracjami pojawia się, gdy zmiany nakładają się na siebie i jest ich zbyt wiele. Wtedy najlepiej usunąć wszystkie stare pliki migracji, tak aby utworzyć jedną, całościową wersję. Jeśli to nie rozwiąże problemu, należy całkowicie usunąć bazę danych z serwera Microsoft SQL (używając **Eksploratora serwera** lub najlepiej **SQL Server Management Studio**). Dopiero wtedy tworzymy jedną nową migrację i wykonujemy aktualizację bazy.



4. Jak udostępnić projekt bez pliku bazy danych?

Aby móc uruchomić projekt u siebie, wystarczą dwa kroki, które wyeliminują konieczność ręcznego tworzenia tabel w SQL.

Krok A: Konfiguracja połączenia (appsettings.json) W pliku konfiguracyjnym definiujemy tzw. *Connection String*. To jedyne miejsce, które użytkownik musi zmienić, aby wskazać swój serwer SQL.



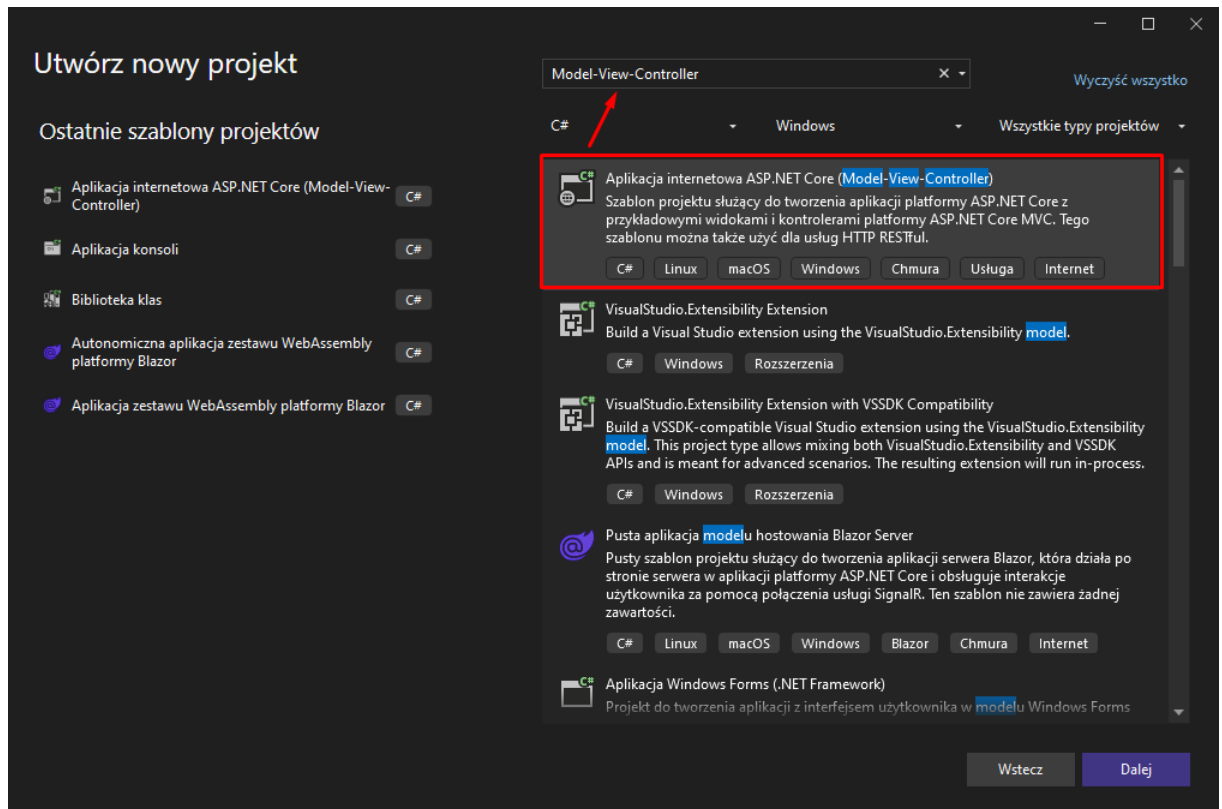
```
1 {
2   "ConnectionStrings": {
3     "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=aspnet-CarRentalSystem-2a3a6095-e78b-40be-b897-2fe4c2ea22b",
4   },
5   "Logging": {
6     "LogLevel": {
7       "Default": "Information",
8       "Microsoft.AspNetCore": "Warning"
9     }
10  },
11  "AllowedHosts": "*"
12 }
13
```

Krok B: Migracje i Snapshot (Migrations) Folder **Migrations** w projekcie zawiera historię zmian w bazie (C# code), a nie samą bazę.

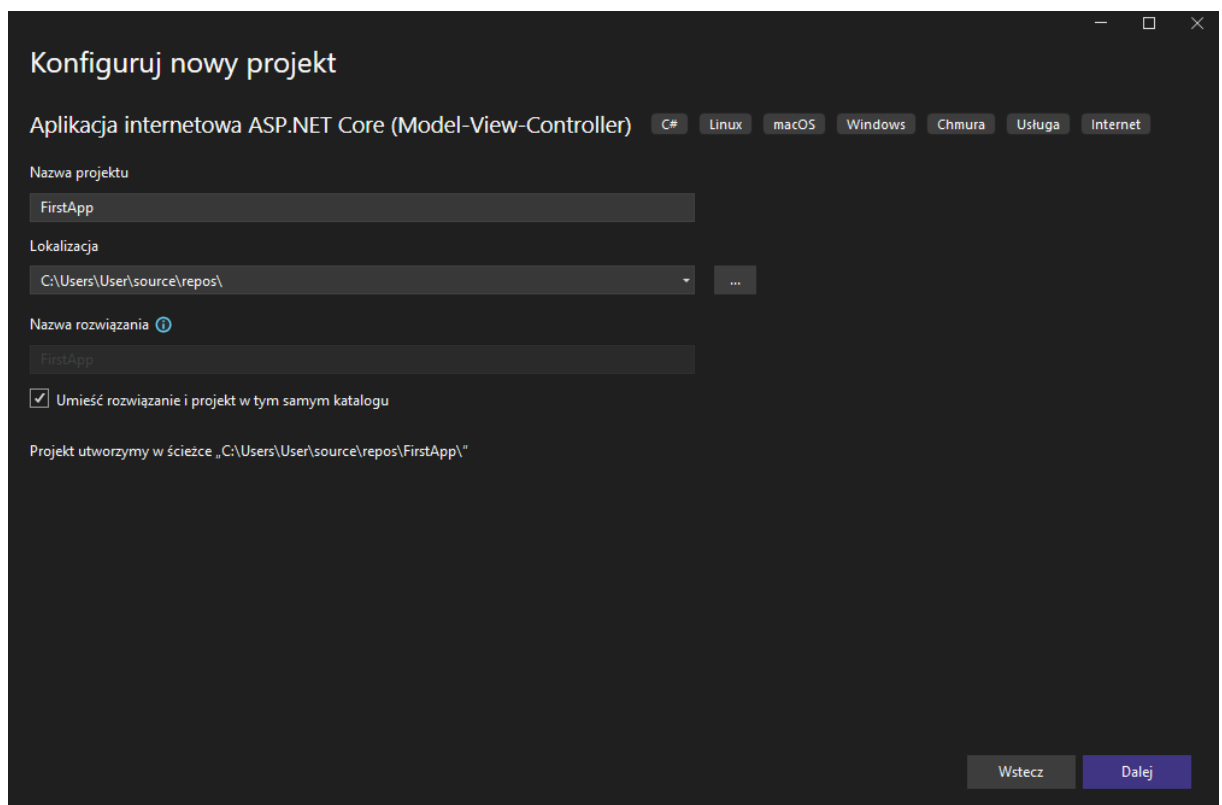
- **Snapshot:** Przechowuje aktualny stan całej bazy.
- **Pliki migracji (np. Init.cs):** Instrukcje "krok po kroku" jak zmodyfikować bazę.
 - Dzięki temu odbiorca projektu musi wpisać w konsoli Package Manager Console tylko jedną komendę: **Update-Database**
 - Entity Framework Core automatycznie odtworzy całą strukturę bazy danych na komputerze odbiorcy na podstawie kodu.

5. Pilotaż stworzenia prostej aplikacji (TUTORIAL)

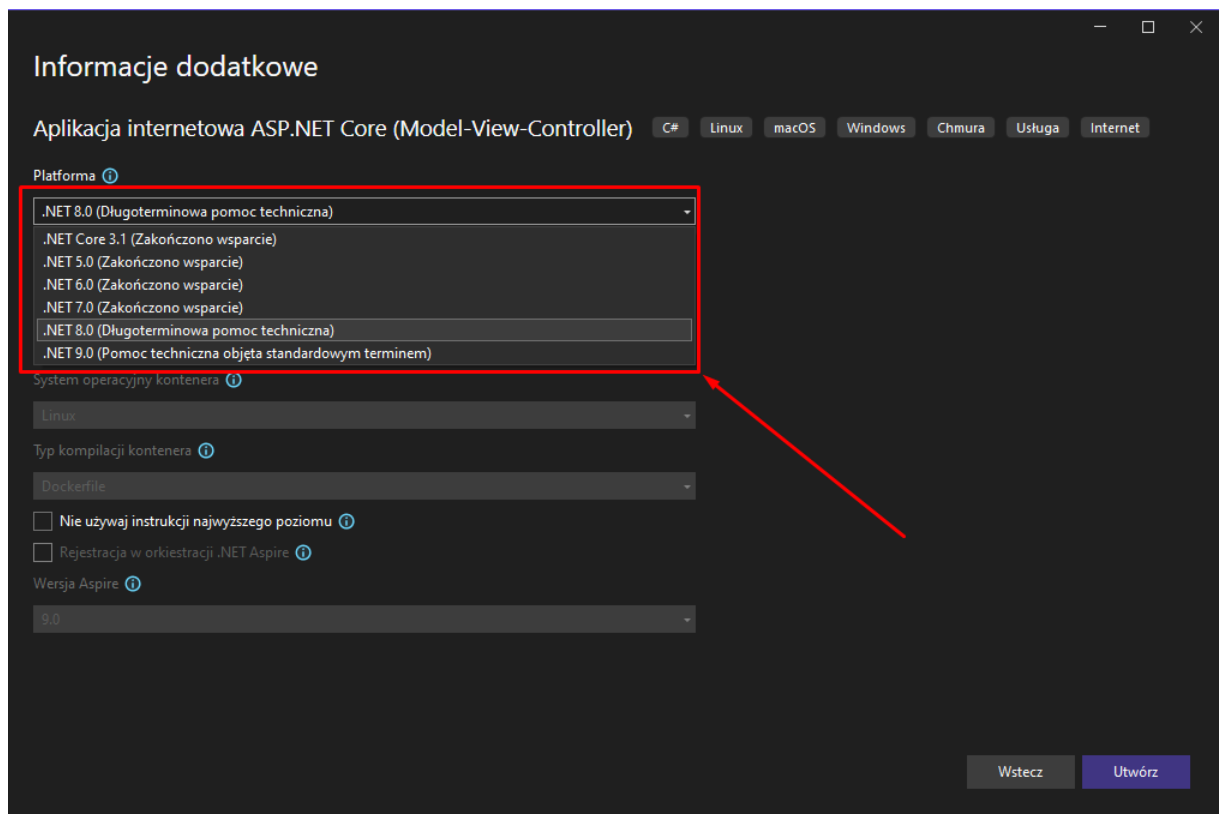
1) Szukamy wzorca MVC (Model-View-Controller)



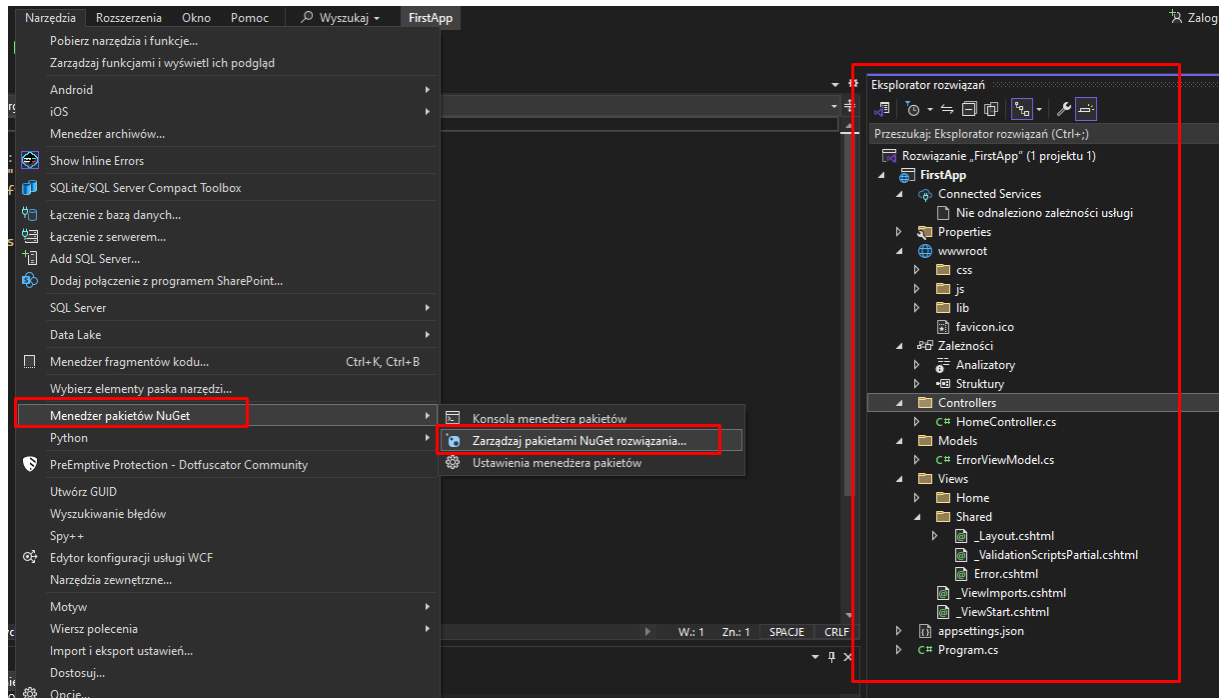
2) Można wedle uznania zaznaczyć opcję umieszczenia rozwiązania i projekt w tym samym katalogu.



3) Wybór platformy .NET



4) Standardowa struktura projektu



5) Instalacja wybranych pakietów, wybrałem .NET 8.0, więc instalując pakiet zwracam uwagę na wersję pozostałych pakietów, które będą doinstalowywane.

NuGet — rozwiązanie - appsettings.json

Przeglądaj Zainstalowane Aktualizacje Konsoliduj Zarządzaj pakietami dla rozwiązania

EntityFrameworkCore x Uwzględnij wersję wstępną Źródło pakietów: nuget.org

Microsoft.EntityFrameworkCore przez aspnet, dotnetframework, EntityFramework, I 10.0.2
Entity Framework Core is a modern object-database mapper for .NET. It supports LINQ queries, change tracking, updates, and schema migrations. EF Core works with SQL Server, Azure SQL D...

Microsoft.EntityFrameworkCore.Abstractions przez aspnet, dotnetframework, E 10.0.2
Provides abstractions and attributes that are used to configure Entity Framework Core

Microsoft.EntityFrameworkCore.Relational przez aspnet, dotnetframework, Entit 10.0.2
Shared Entity Framework Core components for relational database providers.

Microsoft.EntityFrameworkCore.Analyzers przez aspnet, dotnetframework, Entit 10.0.2
CSharp Analyzers for Entity Framework Core.

Microsoft.EntityFrameworkCore.Design przez aspnet, dotnetframework, EntityFra 10.0.2
Shared design-time components for Entity Framework Core tools.

Licencja na każdy pakiet jest udzielana przez jego właściciela. Firma NuGet nie ponosi odpowiedzialności za pakiety innych firm ani nie udziela na nie żadnych licencji.

☐ Nie pokazuj tego więcej

Microsoft.EntityFrameworkCore nuget.org

Wersje: 0

✓	Projekt	Wersja	Zainstalowane	Poziom pakiet
✓	FirstApp			

Zainstalowane: niezainstalowane Odinstaluj

Wersja: 8.0.0 Zainstaluj

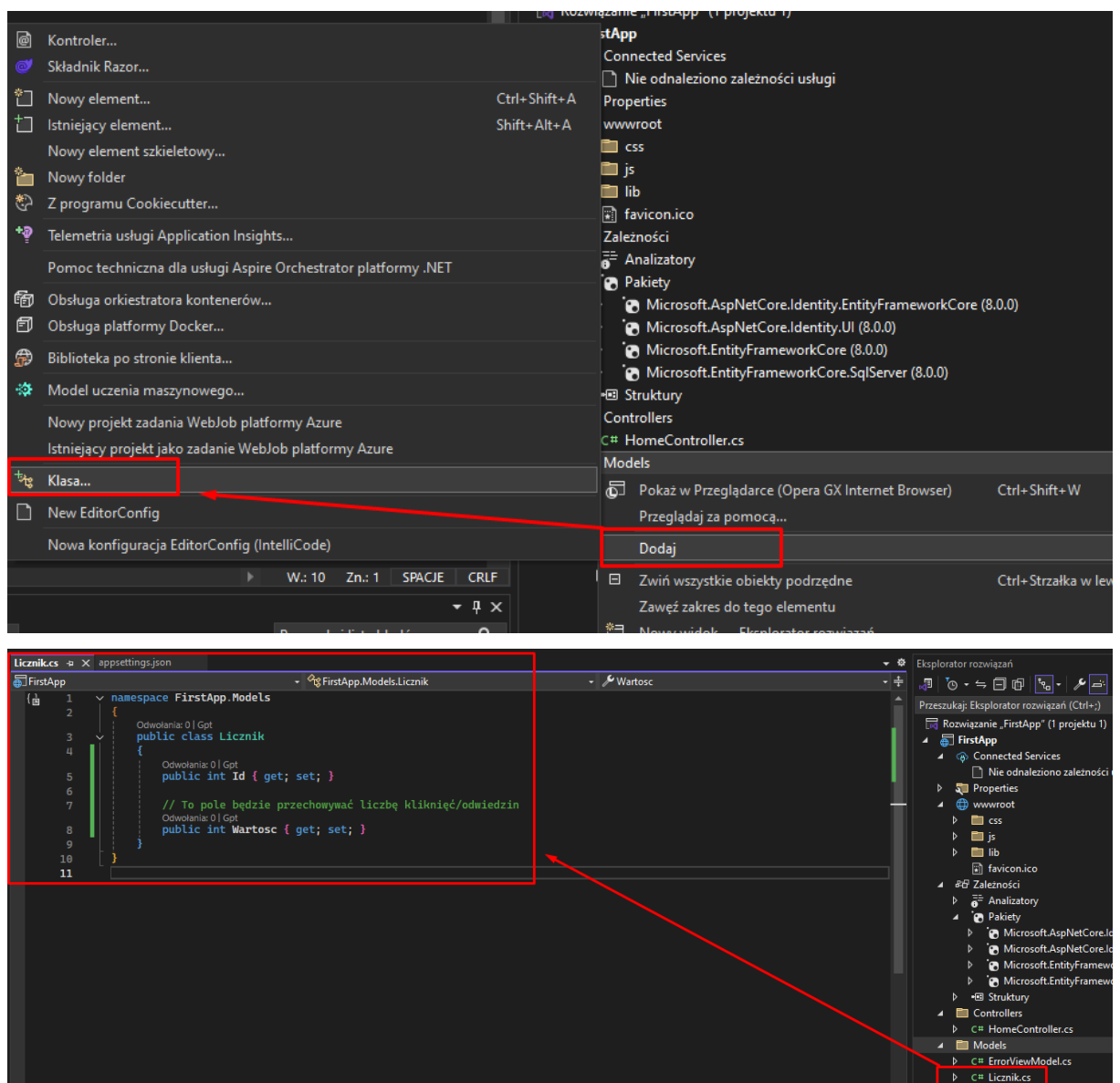
Mapowanie źródła pakietu jest wyłączone. Konfiguruj

Opis

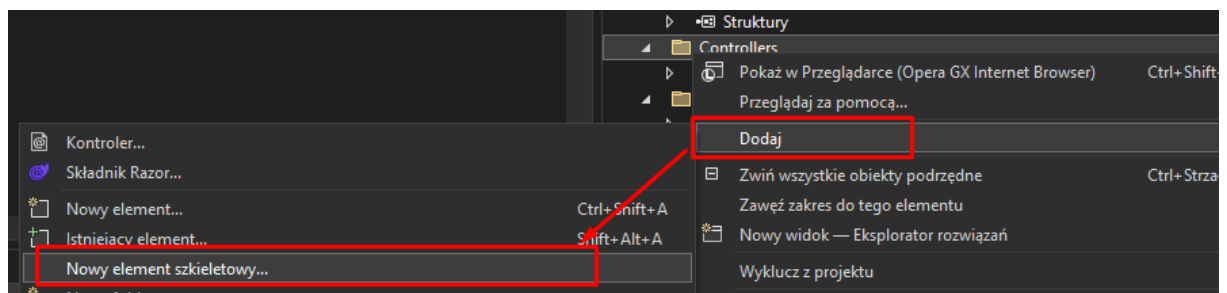
Zależności

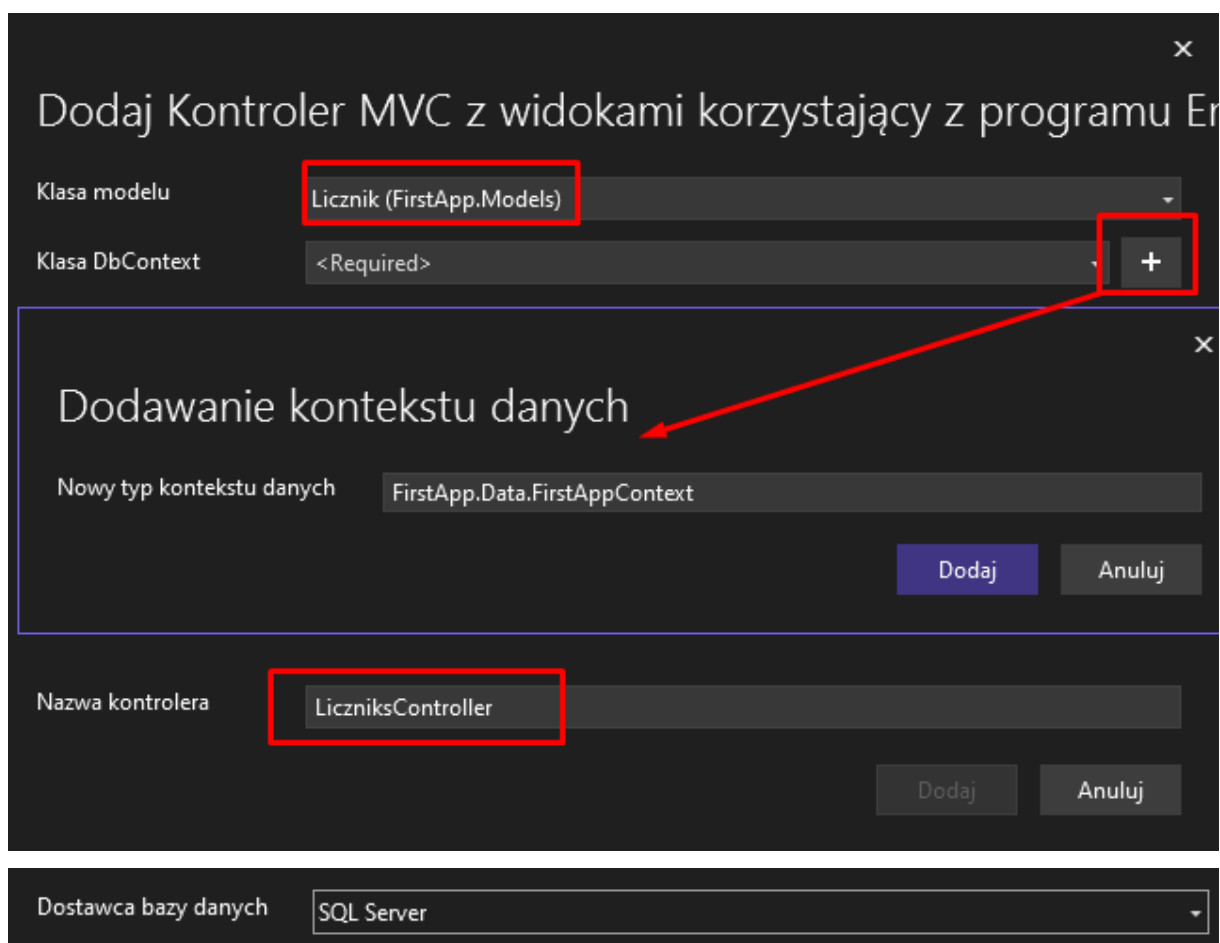
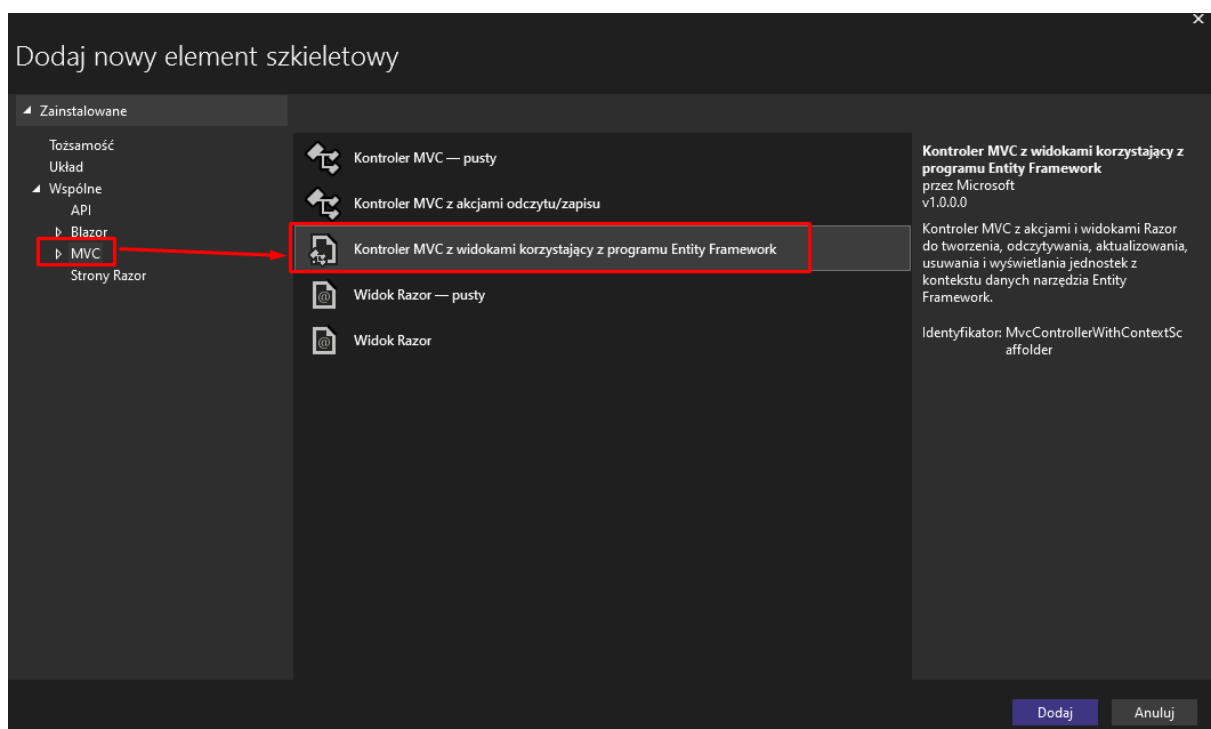
- Analizatory
- Pakiety
 - Microsoft.AspNetCore.Identity.EntityFrameworkCore (8.0.0)
 - Microsoft.AspNetCore.Identity.UI (8.0.0)
 - Microsoft.EntityFrameworkCore (8.0.0)
 - Microsoft.EntityFrameworkCore.SqlServer (8.0.0)

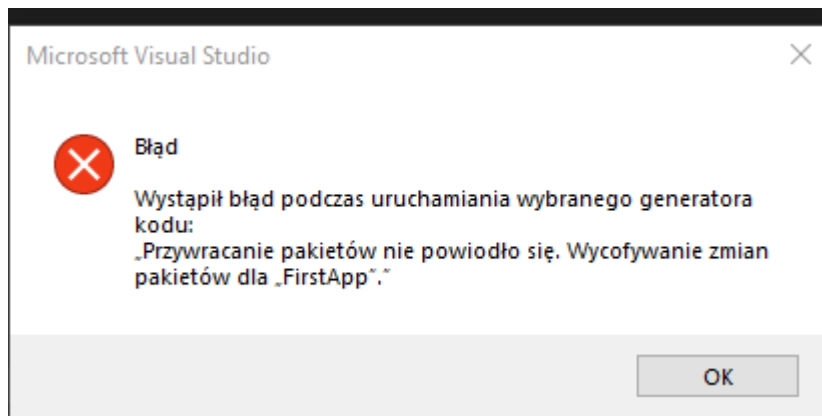
6) Tworzenie przykładowego modelu



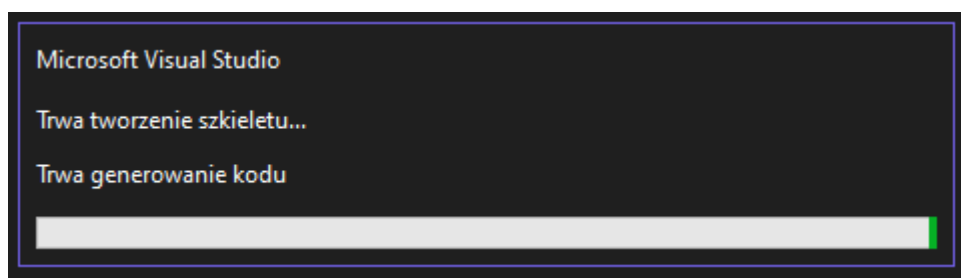
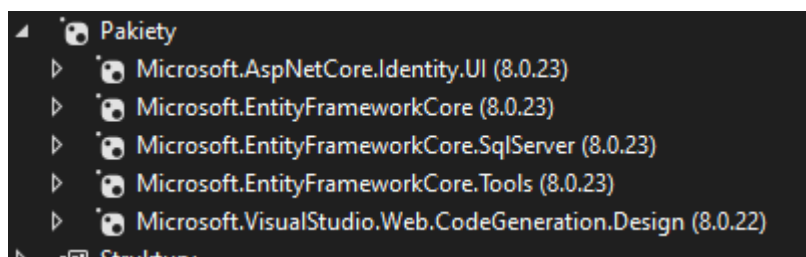
7) Wybieramy „Nowy element szkieletowy”.

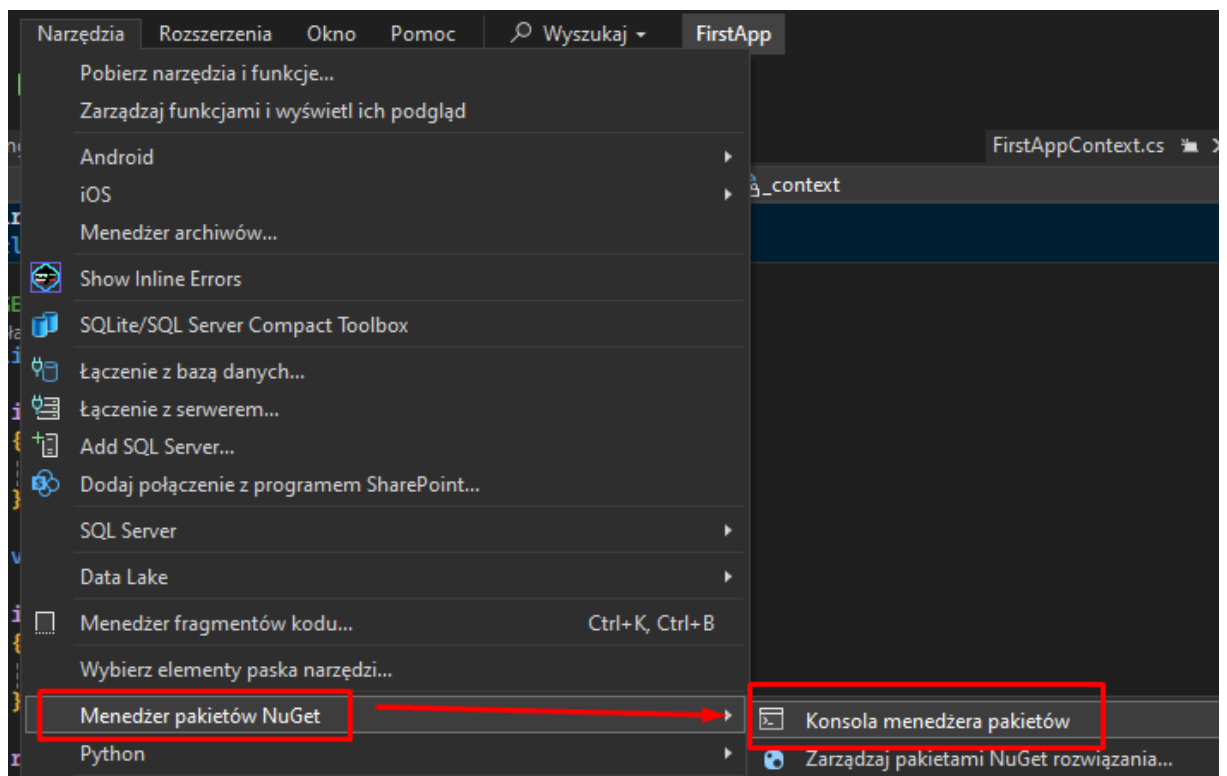
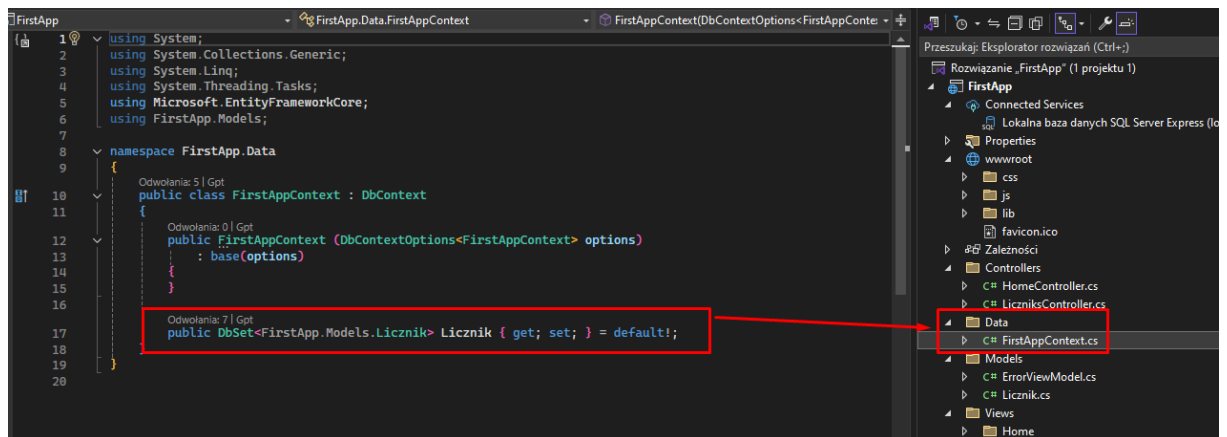
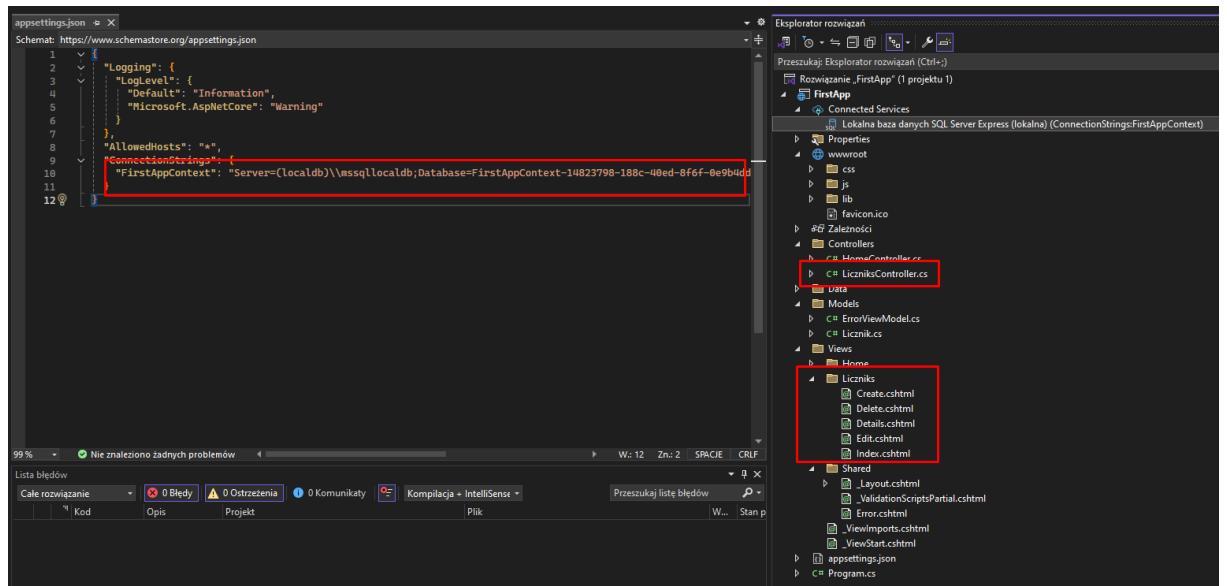


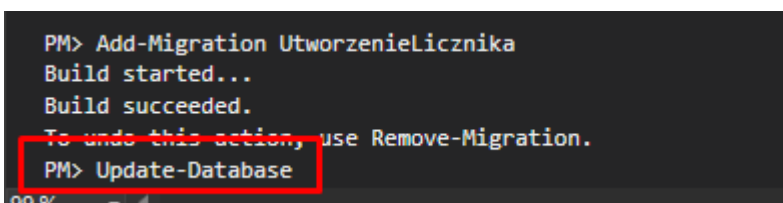
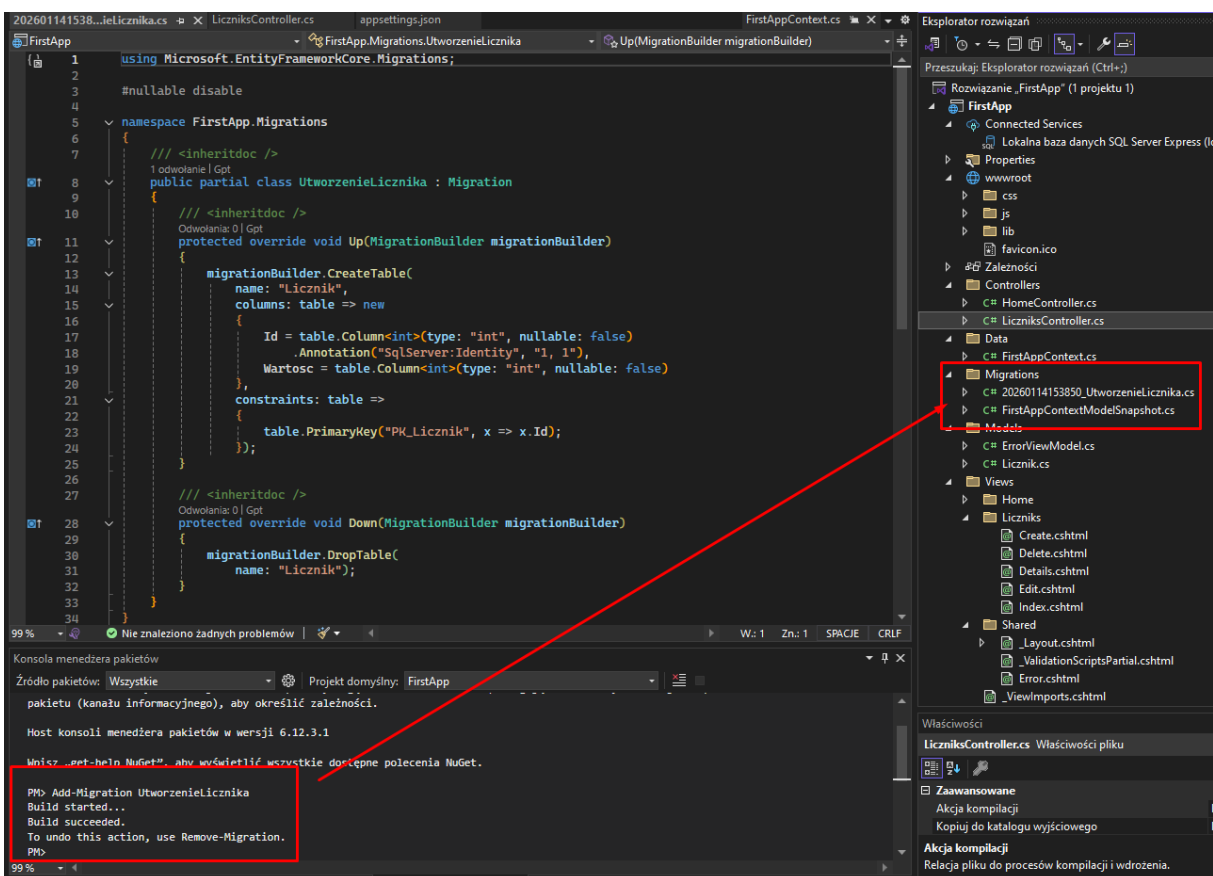
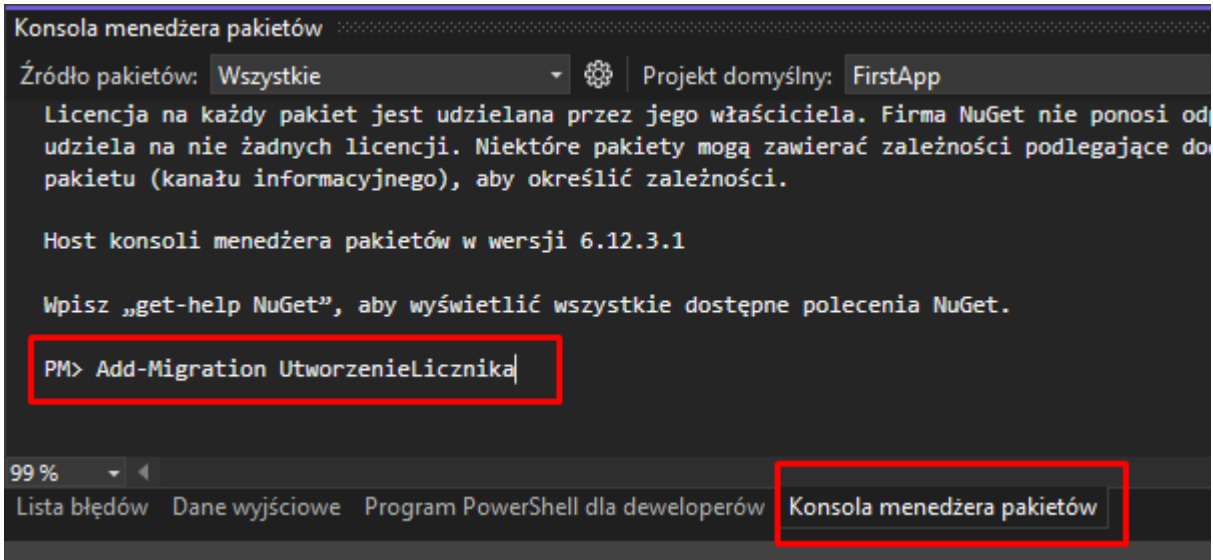




Ten błąd występuje bardzo często i zazwyczaj sygnalizuje braki w pakietach lub konieczność ich aktualizacji. Aby go naprawić, wystarczy w menedżerze NuGet zaktualizować obecne pakiety do nowszej wersji (np. z **8.0.0** do **8.0.23**), a następnie – co bardzo ważne – **zrestartować Visual Studio**. Po ponownym uruchomieniu, proces tworzenia szkieletu (Scaffolding) zajmie chwilę, ale powinien zakończyć się sukcesem bez żadnych komplikacji.







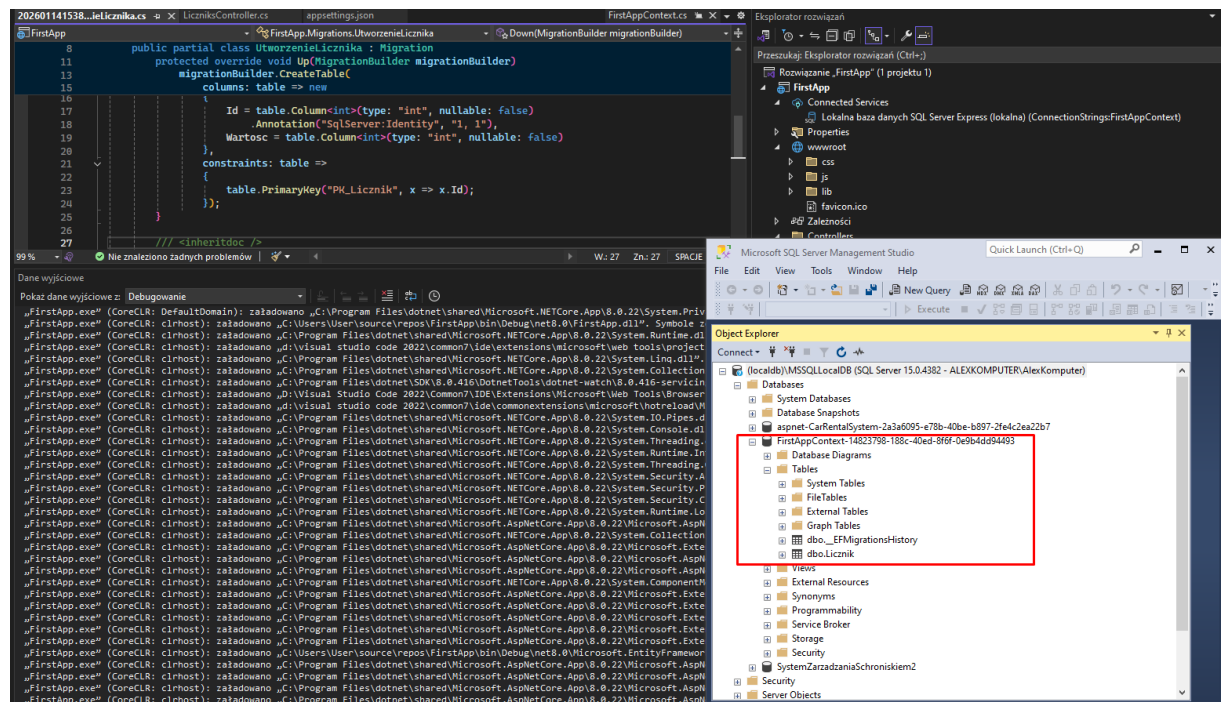
```
Konsola menedżera pakietów
Źródło pakietów: Wszystkie Projekt domyślny: FirstApp

CREATE TABLE [__EFMigrationsHistory] (
    [MigrationId] nvarchar(150) NOT NULL,
    [ProductVersion] nvarchar(32) NOT NULL,
    CONSTRAINT [PK__EFMigrationsHistory] PRIMARY KEY ([MigrationId])
);
Microsoft.EntityFrameworkCore.Database.Command[20101]
  Executed DbCommand (0ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
  SELECT 1
Microsoft.EntityFrameworkCore.Database.Command[20101]
  Executed DbCommand (7ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
  SELECT OBJECT_ID(N'[__EFMigrationsHistory]');
Microsoft.EntityFrameworkCore.Database.Command[20101]
  Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
  SELECT [MigrationId], [ProductVersion]
  FROM [__EFMigrationsHistory]
  ORDER BY [MigrationId];
Microsoft.EntityFrameworkCore.Migrations[20402]
  Applying migration '20260114153850_UtworzenieLicznika'.
Applying migration '20260114153850_UtworzenieLicznika'.
Microsoft.EntityFrameworkCore.Database.Command[20101]
  Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
  CREATE TABLE [Licznik] (
    [Id] int NOT NULL IDENTITY,
    [Wartosc] int NOT NULL,
    CONSTRAINT [PK_Licznik] PRIMARY KEY ([Id])
  );
Microsoft.EntityFrameworkCore.Database.Command[20101]
  Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
  INSERT INTO [__EFMigrationsHistory] ([MigrationId], [ProductVersion])
  VALUES (N'20260114153850_UtworzenieLicznika', N'8.0.23');

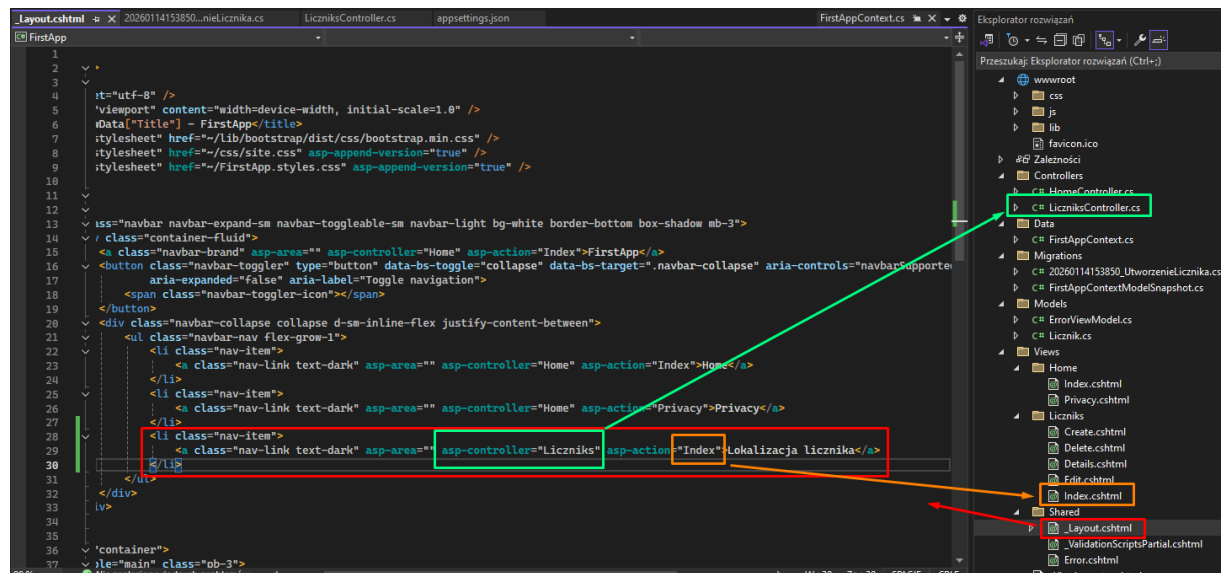
Done.
PM> |
```

Inicjalizacja i weryfikacja bazy danych: Po zakończeniu konfiguracji migracji, należy skompilować i uruchomić projekt (klikając zielony przycisk "  https >  " lub używając klawisza **F5**). Podczas pierwszego uruchomienia aplikacja automatycznie

wykona kod inicjalizujący, co spowoduje **fizyczne utworzenie bazy danych**.



- 8) Teraz musimy sprawić, by nasz licznik był widoczny dla użytkownika. Zamiast umieszczać go na stronie głównej, stworzymy dla niego osobną zakładkę w menu. Przechodzimy do edycji widoku głównego (`_Layout.cshtml`), gdzie dodajemy nowy przycisk. Ważne, aby wskazać mu drogę do naszego kontrolera (**Liczniki**) oraz konkretnego widoku (**Index**), który chcemy wyświetlić..



Po skompilowaniu projektu otrzymujemy domyślny widok typu **CRUD (Create, Read, Update, Delete)**. Choć mechanizm działa i pozwala na ręczne dodawanie rekordów poprzez przycisk **Create**, interfejs ten ma charakter administracyjny (tabela danych). Nie spełnia on jeszcze wymagań naszej aplikacji, która ma opierać

się na interaktywnym przycisku zliczającym, a nie na ręcznym wprowadzaniu wartości..

localhost:7166/Liczniki

FirstApp Home Privacy Lokalizacja licznika

Index

[Create New](#)

Id	Wartosc	
1	1	Edit Details Delete
2	2	Edit Details Delete
3	3	Edit Details Delete

Edit Licznik

Wartosc

[Save](#)

[Back to List](#)

Details Licznik

Wartosc 1

[Edit](#) | [Back to List](#)

Delete Licznik

Are you sure you want to delete this?

Wartosc 3

[Delete](#) | [Back to List](#)

Przykład zapisu do bazy danych

SELECT * FROM dbo.Licznik

Id	Wartosc
1	1
2	2
3	3

Domyślnie mamy **Create, Edit, Details, Delete** opcje, **ALE** można je w każdej chwili usunąć z widoku modyfikując odpowiednio **Index.cshtml** i zastosować w **LiczniksController.cs** odpowiednią implementację działania.

Views/Liczniks/Index.cshtml (domyślny widok)

```
1 @model IEnumerable<FirstApp.Models.Licznik>
2
3 @{
4     ViewData["Title"] = "Index";
5 }
6
7 <h1>Index</h1>
8
9 <p>
10     <a asp-action="Create">Create New</a>
11 </p>
12 <table class="table">
13     <thead>
14         <tr>
15             <th>
16                 @Html.DisplayNameFor(model => model.Wartosc)
17             </th>
18         </tr>
19     </thead>
20     <tbody>
21         @foreach (var item in Model) {
22             <tr>
23                 <td>
24                     @Html.DisplayFor(modelItem => item.Wartosc)
25                 </td>
26                 <td>
27                     <a asp-action="Edit" asp-route-id="@item.Id">Edit</a> |
28                     <a asp-action="Details" asp-route-id="@item.Id">Details</a> |
29                     <a asp-action="Delete" asp-route-id="@item.Id">Delete</a>
30                 </td>
31             </tr>
32         }
33     </tbody>
34 </table>
35
36
```

Views/Liczniks/Index.cshtml (zmodyfikowany widok)

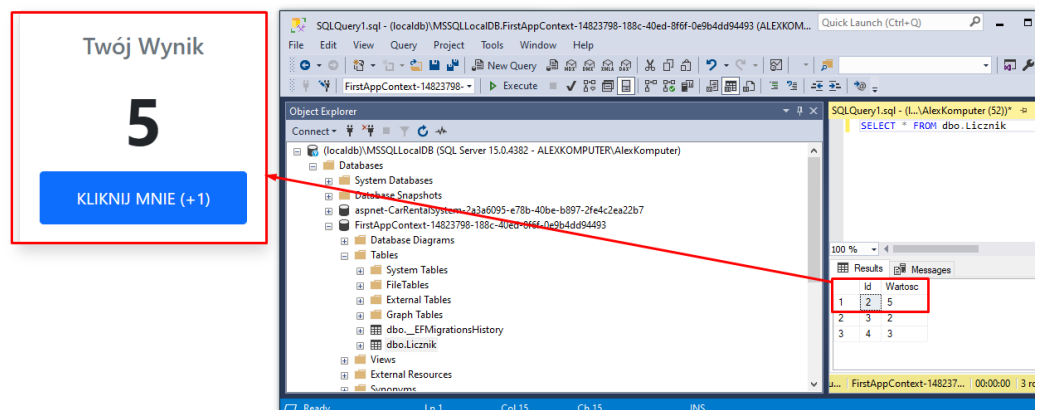
```
1 @model IEnumerable<FirstApp.Models.Licznik>
2
3 @{
4     ViewData["Title"] = "Licznik";
5 }
6
7 <div class="d-flex justify-content-center align-items-center" style="min-height: 60vh;">
8
9     @if (!Model.Any())
10     {
11         <div class="alert alert-warning">
12             Brak liczników w bazie. Musisz dodać chociaż jeden w bazie danych.
13         </div>
14     }
15     else
16     {
17         @foreach (var item in Model)
18         {
19             <div class="card shadow text-center p-4 m-3" style="min-width: 300px;">
20
21                 <h3 class="text-muted">Twój Wynik</h3>
22
23                 <div class="display-1 fw-bold my-3">
24                     @item.Wartosc
25                 </div>
26
27                 <form asp-action="Zwiększ" asp-route-id="@item.Id" method="post">
28                     <button type="submit" class="btn btn-primary btn-lg w-100 py-3">
29                         KLIKNIJ MNIE (+1)
30                     </button>
31                 </form>
32
33             </div>
34         }
35     }
36 </div>
37
```

Implementacja w kontrolerze **LiczniksController.cs** funkcjonalności.

```
10
11 namespace FirstApp.Controllers
12 {
13     1 odwołanie | Gpt
14     public class LiczniksController : Controller
15     {
16         private readonly FirstAppContext _context;
17
18         Odwołania: 0 | Gpt
19         public LiczniksController(FirstAppContext context)
20         {
21             _context = context;
22         }
23
24         // GET: Liczniks
25         Odwołania: 4 | Gpt
26         public async Task<IActionResult> Index()
27         {
28             return View(await _context.Licznik.ToListAsync());
29         }
30
31         [HttpPost]
32         Odwołania: 0 | Gpt
33         public async Task<IActionResult> Zwiększ(int id)
34         {
35             var licznik = await _context.Licznik.FindAsync(id);
36
37             if (licznik != null)
38             {
39                 licznik.Wartosc++; // Zwiększamy o 1
40                 await _context.SaveChangesAsync(); // Zapisujemy w bazie
41             }
42
43             return RedirectToAction(nameof(Index)); // Wracamy do widoku
44         }
45
46         // GET: Liczniks/Details/5
47         Odwołania: 0 | Gpt
48     }
49 }
```

Efekt poprawnie działającego licznika i zapis do bazy również działa 😊.

Lokalizacja licznika

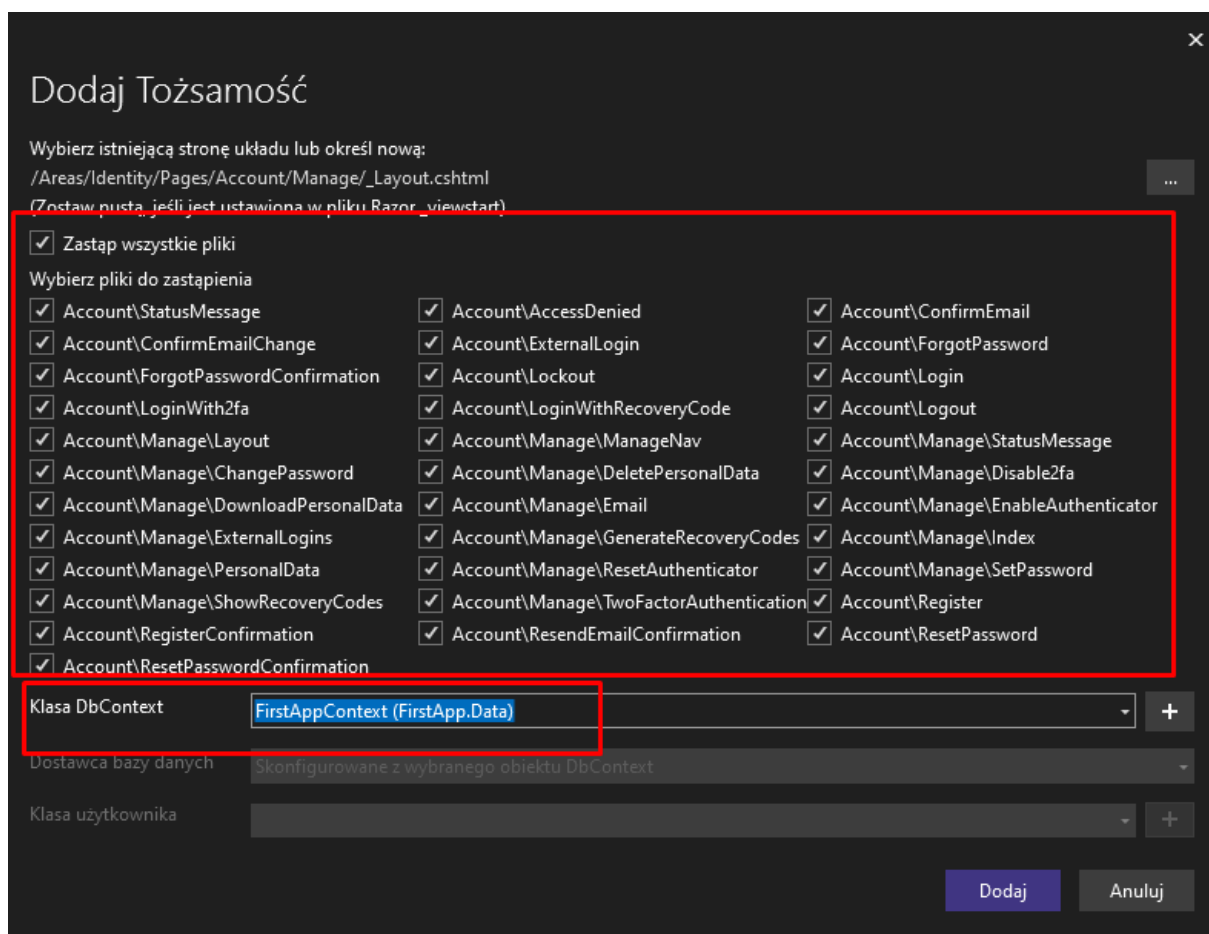
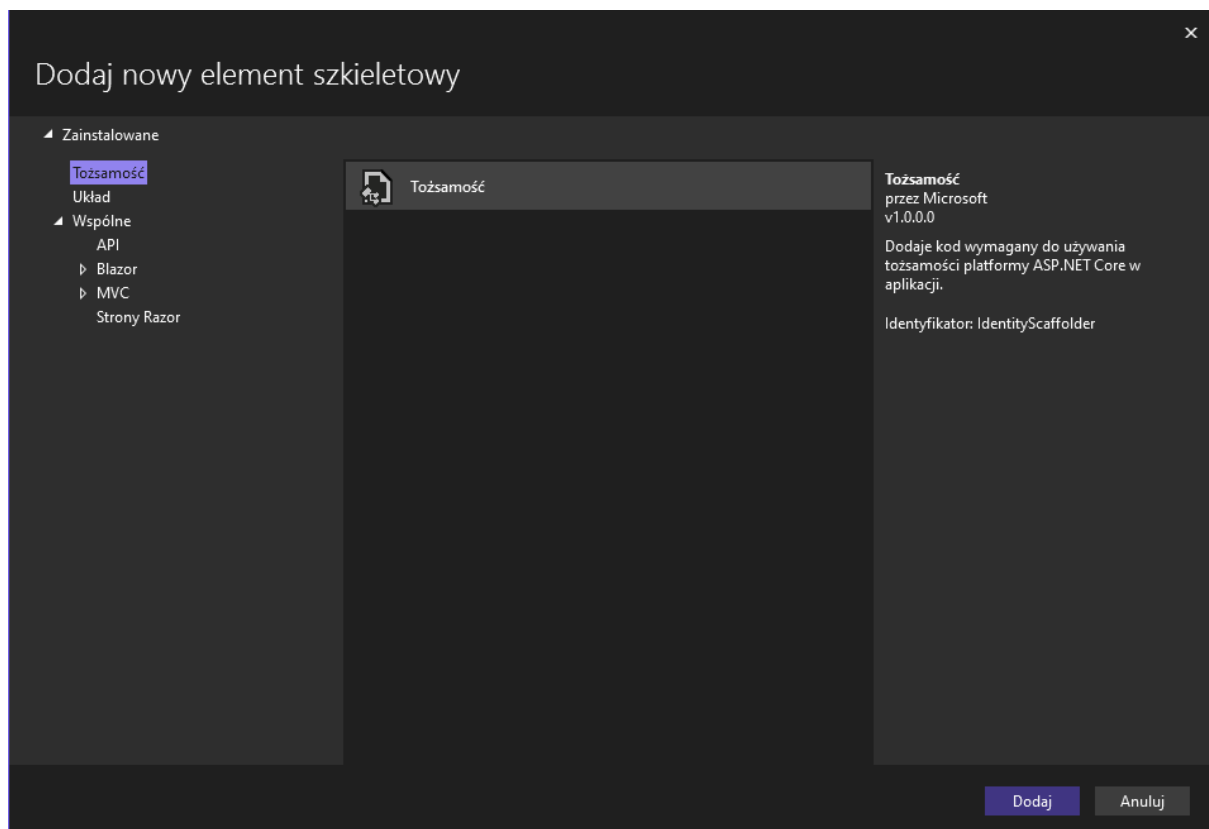


- 9) Adaptacja kontekstu danych do obsługi tożsamości (Identity). Aby zintegrować pakiet **ASP.NET Core Identity** z istniejącą bazą danych, konieczna jest modyfikacja klasy kontekstu (**FirstAppContext.cs**). Standardowa klasa DbContext nie posiada definicji tabel użytkowników.

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Threading.Tasks;
5  using Microsoft.EntityFrameworkCore;
6  using FirstApp.Models;
7
8  namespace FirstApp.Data
9  {
10     public class FirstAppContext : DbContext
11     {
12         public FirstAppContext(DbContextOptions<FirstAppContext> options)
13             : base(options)
14         {
15         }
16
17         public DbSet<FirstApp.Models.Licznik> Licznik { get; set; } = default!;
18     }
19 }
20
```

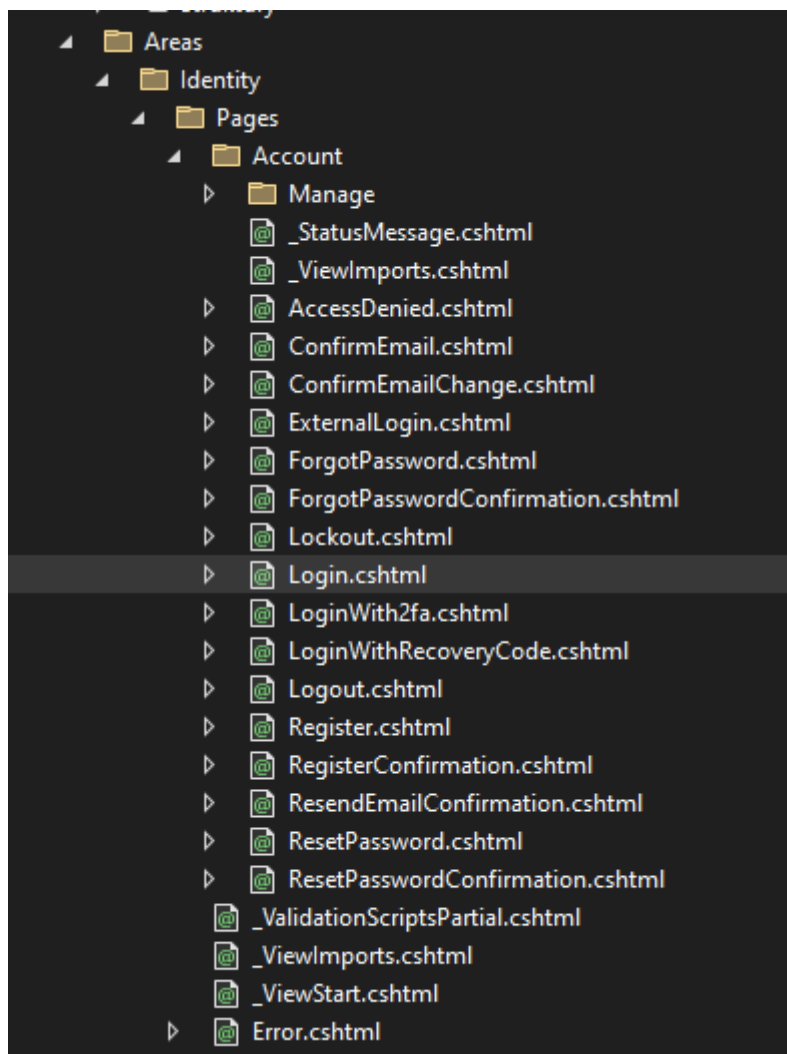
```
rozwiązanie  FirstAppContext.cs  Licznik.cs  Index.cshtml  _Layout.cshtml  20260114153850...nieLicznika.cs
pp  FirstApp.Data.FirstAppContext  Licznik
1  using FirstApp.Models;
2  using Microsoft.EntityFrameworkCore;
3  using Microsoft.AspNetCore.Identity.EntityFrameworkCore; // <--- 1. Dajemy tę dyrektywę using
4
5  namespace FirstApp.Data
6  {
7      // 2. Zmieniamy 'DbContext' na 'IdentityDbContext'
8      public class FirstAppContext : IdentityDbContext
9      {
10         public FirstAppContext(DbContextOptions<FirstAppContext> options)
11             : base(options)
12         {
13         }
14
15         public DbSet<Licznik> Licznik { get; set; }
16     }
17 }
```

- 10) Potem można kliknąć PPM na cały projekt, wybrać opcję **Dodaj**, a następnie **Nowy element szkieletowy**. Z dostępnych szablonów wybieramy pozycję **Tożsamość (Identity)**, co pozwoli na automatyczne dodanie do aplikacji gotowych stron rejestracji, logowania oraz zarządzania kontem użytkownika.



Czasami do poprawnego działania wymagany jest dodatkowy pakiet

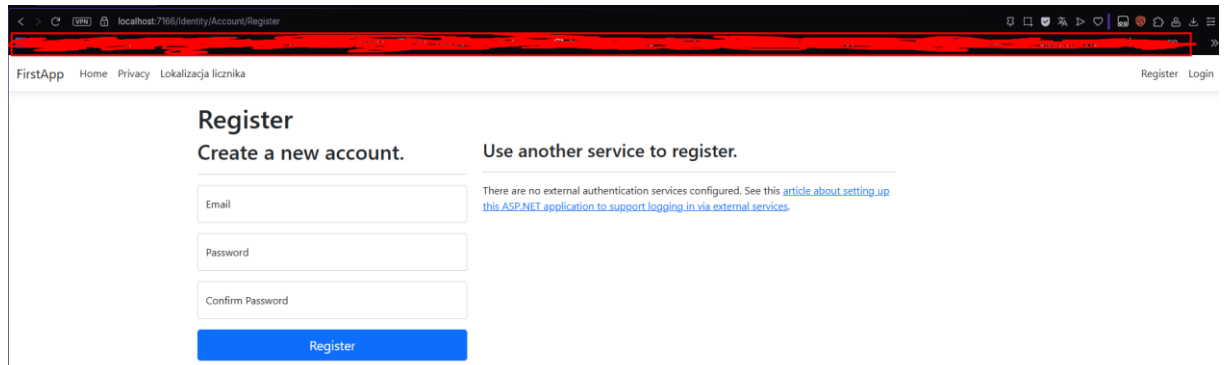
Microsoft.EntityFrameworkCore.Sqlite. Przeważnie program powinien pobrać go samoczynnie, jednak w razie problemów trzeba go zainstalować ręcznie przez **menedżer NuGet**. Gdy pakiet znajdzie się w projekcie, nasza struktura folderów i plików Identity pojawi się w prawidłowej formie.



Na sam koniec musimy sprawić, by opcje logowania i rejestracji były rzeczywiście widoczne dla użytkowników. Dodajemy je do głównego pliku widoku (**_Layout.cshtml**). Dodatkowo, w pliku konfiguracyjnym **Program.cs** musimy włączyć obsługę tzw. **Razor Pages**. Jest to niezbędne, ponieważ właśnie tam znajdują się wszystkie formularze, które pozwalają użytkownikom na zakładanie kont i logowanie się do naszej aplikacji.

```
Program.cs
1 using Microsoft.EntityFrameworkCore;
2 using Microsoft.Extensions.DependencyInjection;
3 using FirstApp.Data;
4 using Microsoft.AspNetCore.Identity;
5 var builder = WebApplication.CreateBuilder(args);
6 builder.Services.AddDbContext<FirstAppContext>(options =>
7     options.UseSqlServer(builder.Configuration.GetConnectionString("FirstAppContext")) ?? throw new InvalidOperationException("Connect
8
9     builder.Services.AddDefaultIdentity<IdentityUser>(options => options.SignIn.RequireConfirmedAccount = true).AddEntityFrameworkStores<
10
11 // Add services to the container.
12 builder.Services.AddControllersWithViews();
13
14 builder.Services.AddRazorPages(); // <--- DODAC DO DO OBSŁUGI RAZOR PAGES /Formularze z Identity
15
16 var app = builder.Build();
17
18 // Configure the HTTP request pipeline.
19 if (!app.Environment.IsDevelopment())
20 {
21     app.UseExceptionHandler("/Home/Error");
22     // The default HSTS value is 30 days. You may want to change this for production scenarios, see https://aka.ms/aspnetcore-hsts.
23     app.UseHsts();
24 }
25
26 app.UseHttpsRedirection();
27 app.UseStaticFiles();
28
29 app.UseRouting();
30
31 app.UseAuthentication(); // <--- Dodać to do bo sprawdza najpierw kim jesteśmy przed Authorization
32 app.UseAuthorization();
33
34 app.MapControllerRoute(
35     name: "default",
36     pattern: "{controller=Home}/{action=Index}/{id?}");
37
38 app.MapRazorPages(); // <--- Dodać to do możliwości uruchomienia routingu dla stron Identity
39
40 app.Run();
```

```
FirstApp
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="utf-8" />
5 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6 <title>@ViewData["Title"] - FirstApp</title>
7 <link rel="stylesheet" href="/lib/bootstrap/dist/css/bootstrap.min.css" />
8 <link rel="stylesheet" href="/css/site.css" asp-append-version="true" />
9 <link rel="stylesheet" href="/FirstApp.styles.css" asp-append-version="true" />
10 </head>
11 <body>
12 <header>
13 <nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-light bg-white border-bottom box-shadow mb-3">
14 <div class="container-fluid">
15 <a class="navbar-brand" asp-area="" asp-controller="Home" asp-action="Index">FirstApp</a>
16 <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target=".navbar-collapse" aria-controls="
17     aria-expanded="false" aria-label="Toggle navigation">
18 <span class="navbar-toggler-icon"></span>
19 </button>
20 <div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
21 <ul class="navbar-nav flex-grow-1">
22 <li class="nav-item">
23 <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Index">Home</a>
24 </li>
25 <li class="nav-item">
26 <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Privacy">Privacy</a>
27 </li>
28 <li class="nav-item">
29 <a class="nav-link text-dark" asp-area="" asp-controller="Liczniki" asp-action="Index">Lokalizacja licznika
30 </li>
31 </ul>
32 <partial name="_LoginPartial" />
33 </div>
34 </header>
35 <div class="container">
36
37 Nie znaleziono żadnych problemów
```



Log in

Use a local account to log in.

Email

Password

☐ Remember me?

Log in

[Forgot your password?](#)

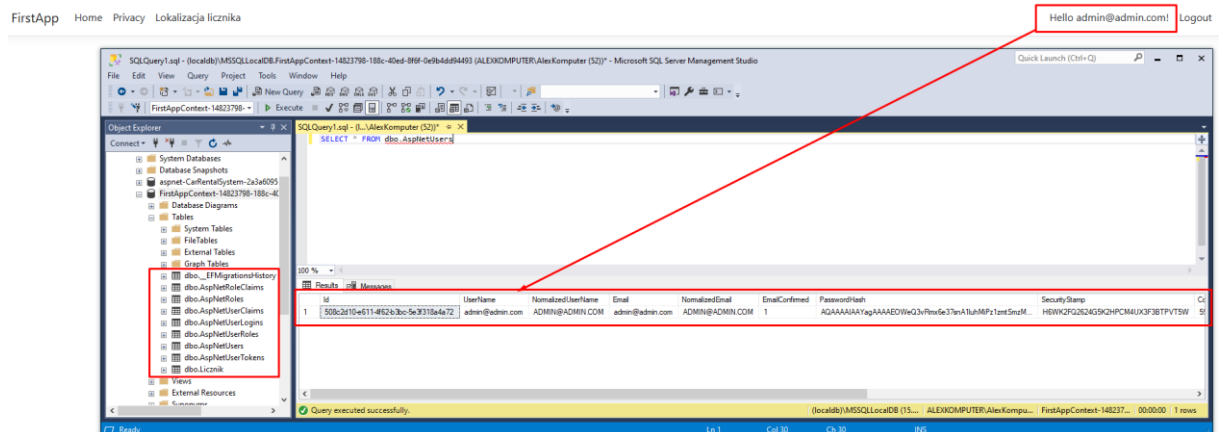
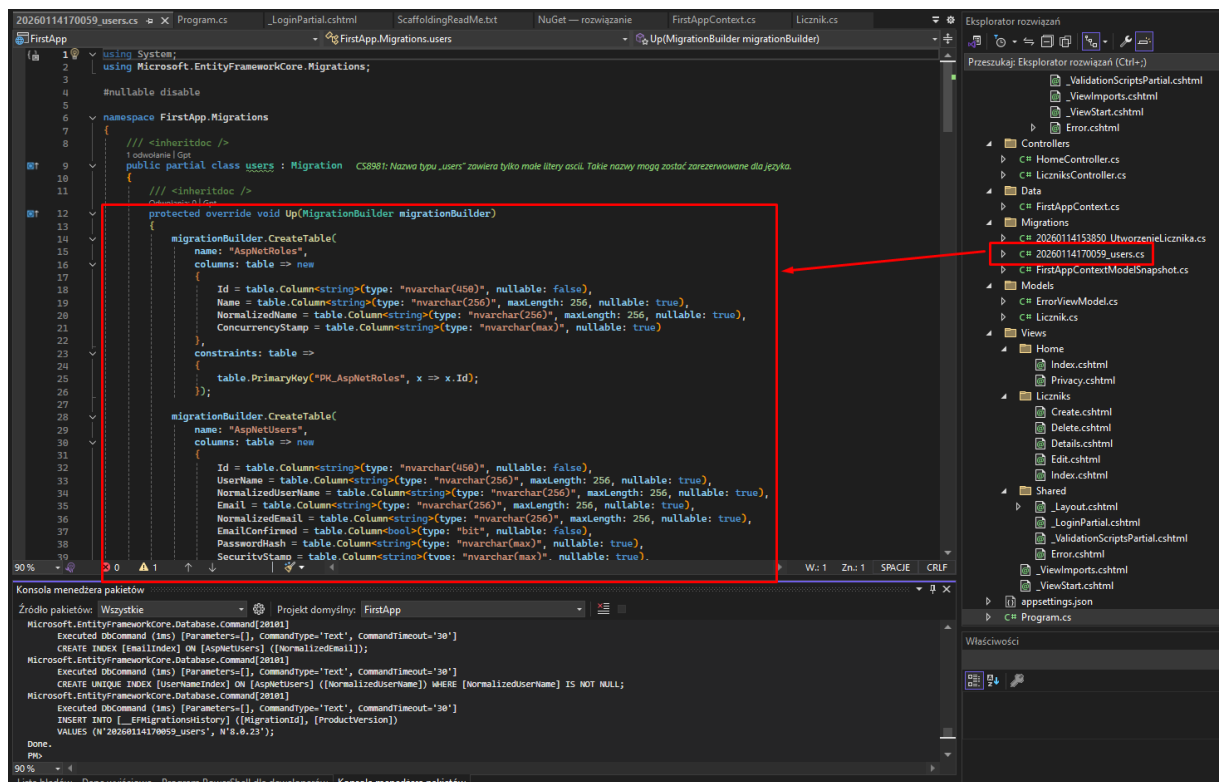
[Register as a new user](#)

[Resend email confirmation](#)

Use another service to log in.

There are no external authentication services configured. See this [article about setting up this ASP.NET application to support logging in via external services](#).

UWAGA! Jeśli chcemy uzyskać możliwość tworzenia kont, musimy najpierw przeprowadzić nową migrację dla systemu Identity oraz wykonać aktualizację bazy danych (Update). Gdy te operacje zakończą się sukcesem, należy ponownie skompilować projekt. Dopiero wtedy będziemy w stanie zarejestrować się i zalogować do naszej aplikacji.



6. Etap rozbudowywania istniejącej aplikacji na podstawie **CarRentalSystem** – system służący do wypożyczania samochodów. [Kody na potrzebę zajęć]

<https://github.com/chomiczo/CarRentalSystem-3.1>

1) Dodawanie nowej funkcjonalności **Zgłaszanie Szkody Samochodu**.

2) Klasa DamageReport.cs

```

public class DamageReport {
    public int Id { get; set; }
    public string? Description { get; set; }
    public DateTime? ReportDate { get; set; }
    public int CarModelId { get; set; } //klucz obcy
}

```

```
public CarModel? Car { get; set; }
}
```

3) Views/DamageReports/Delete.cshtml

```
<dd class = "col-sm-10">
    @Html.DisplayFor(model => model.Car.Brand)
</dd>
```

4) Views/DamageReports/Details.cshtml

```
<dd class = "col-sm-10">
    @Html.DisplayFor(model => model.Car.Brand)
</dd>
```

5) Views/DamageReports/Index.cshtml

```
@model IEnumerable<CarRentalSystem.Models.DamageReport>
@{
    ViewData["Title"] = "Lista Szkód";
}
<h1>Index - Zarządzanie Szkodami</h1>
<div class="row mb-3">
    <div class="col-md-6">
        <p>
            <a asp-action="Create" class="btn btn-success">Dodaj nową szkodę</a>
        </p>
    </div>
    @* --- TUTAJ DODAJEMY FORMULARZ WYSZUKIWANIA --- *@
    <div class="col-md-6 text-right">
        <form asp-action="Index" method="get" class="form-inline float-right">
            <div class="input-group">
                <input type="text" name="searchString" class="form-control"
                    placeholder="Szukaj w opisie..." />
                <div class="input-group-append">
                    <button type="submit" class="btn btn-primary">Filtruj</button>
                </div>
            </div>
        </form>
    </div>
</div>
```

```

</div>
<table class="table table-striped">
  <thead>
    <tr>
      <th>@Html.DisplayNameFor(model => model.Description)</th>
      <th>@Html.DisplayNameFor(model => model.ReportDate)</th>
      <th>@Html.DisplayNameFor(model => model.Car)</th>
      <th>Akcje</th>
    </tr>
  </thead>
  <tbody>
    @foreach (var item in Model)
    {
      <tr>
        <td>@Html.DisplayFor(modelItem => item.Description)</td>
        <td>@Html.DisplayFor(modelItem => item.ReportDate)</td>
        <td>@Html.DisplayFor(modelItem => item.Car.Brand) </td>
        <td>
          <a asp-action="Edit" asp-route-id="@item.Id" class="btn btn-sm btn-info">Edit</a>
          <a asp-action="Details" asp-route-id="@item.Id" class="btn btn-sm btn-warning">Details</a>
          <a asp-action="Delete" asp-route-id="@item.Id" class="btn btn-sm btn-danger">Delete</a>
        </td>
      </tr>
    }
  </tbody>
</table>

```

6) Shared/_Layout.cshtml

```
<li class="nav-item">
    <a class="nav-link text-dark" asp-controller="DamageReports" asp-
action="Index">Zgłoszenia szkód</a>
</li>
```

7) Controllers/DamageReportsController.cs – Edit (GET)

```
// GET: DamageReports/Edit/5
public async Task<IActionResult> Edit(int? id)
{
    if (id == null || _context.DamageReport == null)
    {
        return NotFound();
    }

    var damageReport = await _context.DamageReport.FindAsync(id);
    if (damageReport == null)
    {
        return NotFound();
    }

    // Kluczowe: przekazujemy "Brand" do listy rozwijanej
    ViewData["CarModelId"] = new SelectList(_context.CarModel, "Id",
"Brand", damageReport.CarModelId);
    return View(damageReport);
}
```

8) Controllers/DamageReportsController.cs – Edit (POST)

```
// POST: DamageReports/Edit/5
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(int id,
[Bind("Id,Description,ReportDate,CarModelId")]
CarRentalSystem.Models.DamageReport damageReport)
{
    if (id != damageReport.Id)
    {

```

```

        return NotFound();
    }

    // Kluczowe: ignorujemy obiekt nawigacyjny, by walidacja przeszła
    ModelState.Remove("Car");

    if (ModelState.IsValid)
    {
        try
        {
            _context.Update(damageReport);
            await _context.SaveChangesAsync();
        }
        catch (DbUpdateConcurrencyException)
        {
            if (!DamageReportExists(damageReport.Id))
            {
                return NotFound();
            }
            else
            {
                throw;
            }
        }
        return RedirectToAction(nameof(Index));
    }

    // Jeśli walidacja się nie uda, ponownie ładujemy listę marek (Brand)
    ViewData["CarModelId"] = new SelectList(_context.CarModel, "Id",
"Brand", damageReport.CarModelId);
    return View(damageReport);
}

```

9) Controllers/DamageReportsController.cs – Create (GET)

```
// GET: DamageReports/Create
```

```

public IActionResult Create()
{
    ViewData["CarModelId"] = new SelectList(_context.CarModel, "Id",
"Brand");
    return View();
}

```

10) Controllers/DamageReportsController.cs – Create (POST)

```

// POST: DamageReports/Create
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult>
Create([Bind("Id,Description,ReportDate,CarModelId")]
CarRentalSystem.Models.DamageReport damageReport)
{
    ModelState.Remove("Car");

    if (ModelState.IsValid)
    {
        _context.Add(damageReport);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }

    ViewData["CarModelId"] = new SelectList(_context.CarModel, "Id",
"Brand", damageReport.CarModelId);
    return View(damageReport);
}

```

11) Controllers/DamageReportsController.cs – Index

```

// GET: DamageReports
public async Task<IActionResult> Index(string searchString)
{
    var reports = _context.DamageReport.Include(d => d.Car).AsQueryable();

    if (!string.IsNullOrEmpty(searchString))

```

```
{  
    reports = reports.Where(s => s.Description.Contains(searchString));  
}  
  
return View(await reports.ToListAsync());  
}
```